

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch12_nn_fundamentals

AYuSh

Contents

Chapter 12: Neural Network Fundamentals	3
---	---

Chapter 12: Neural Network Fundamentals

Everything in deep learning is elaboration on this chapter's machinery: layers of linear maps and nonlinearities, trained by gradient descent, with gradients delivered by backpropagation. Interviewers probe here mercilessly because the material is checkable — "derive backprop for a two-layer network" has a right answer, and hand-waving is instantly visible. The chapter's promise is that nothing stays abstract: the perceptron's XOR failure is run, backprop is derived symbol-by-symbol and then verified against numerical gradients to nine decimal places, each activation's saturation zone is measured, each loss's gradient behavior is staged, and the initialization rules are demonstrated by watching signals live or die through twenty layers.

The route: the perceptron and its limits, the MLP that transcends them, and what the universal approximation theorem does and does not promise (Listing 1, Listing 7). Forward and backward passes derived by hand and gradient-checked (Listing 2). Activation functions — sigmoid, tanh, ReLU, Leaky ReLU, GELU, Swish — with their gradients and failure zones measured (Listing 3). Loss functions — MSE, cross-entropy, hinge, focal, contrastive, triplet — with the staged demonstration of why cross-entropy pairs with sigmoid/softmax and what focal loss does to easy examples (Listing 4). Weight initialization — why zeros fail, what Xavier and He preserve (Listing 5). Vanishing and exploding gradients measured layer-by-layer, plus gradient clipping (Listing 6). The finale trains the from-scratch MLP to 96% on real digit data (Listing 8).

The perceptron, the MLP, and universal approximation

The perceptron (Rosenblatt, 1958) is a single thresholded linear unit: output $\hat{y} = 1$ if $w^T x + b > 0$ else 0. Its learning rule — on each mistake, nudge w by $\eta(y - \hat{y})x$ — is guaranteed to converge *if the classes are linearly separable* (the perceptron convergence theorem). That conditional is the famous trap: XOR is not linearly separable — no line puts $(0, 1)$ and $(1, 0)$ on one side and $(0, 0)$, $(1, 1)$ on the other — and Listing 1 shows the perceptron nailing AND and OR at accuracy 1.00 and flatlining at 0.50 (coin-flip) on XOR forever. Minsky and Papert's 1969 formalization of this limit helped freeze neural network research for a decade; the answer, hidden layers, was known in principle but not trainable until backpropagation was popularized in 1986.

The multilayer perceptron (MLP) stacks layers: $h = \phi(W_1 x + b_1)$, $\hat{y} = \psi(W_2 h + b_2)$, with ϕ a *nonlinear* activation. The nonlinearity is load-bearing: composing linear maps yields a linear map ($W_2(W_1 x) = (W_2 W_1)x$), so without ϕ a hundred layers collapse to one. With it, the hidden layer re-represents the input — for XOR, two tanh units carve two half-planes whose combination is linearly separable to the output unit, and Listing 1's

2-2-1 network hits accuracy 1.00 with confident probabilities (0.001/0.999). That is the mental model to carry: *hidden layers learn a representation in which the problem becomes linear*.

The universal approximation theorem (Cybenko 1989; Hornik 1991): a feedforward network with *one* hidden layer and enough units can approximate any continuous function on a compact set to arbitrary accuracy, for any non-polynomial activation. Listing 7 makes it concrete — fitting a wiggly, discontinuous target, MSE falls $0.121 \rightarrow 0.022 \rightarrow 0.007$ as width grows $2 \rightarrow 8 \rightarrow 32$. The theorem is routinely over-quoted, so state its three silences: it promises **existence**, not that gradient descent will *find* the approximating weights; it says nothing about **how many** units are needed (the required width can grow exponentially in input dimension); and it says nothing about **generalization** from finite samples. Depth's practical advantage — some functions representable with polynomial-size deep networks require exponential-size shallow ones — is why the field went deep rather than wide.

Forward and backward: deriving backpropagation by hand

Set up a two-layer classifier precisely; every later architecture is this pattern with more layers. Inputs $X \in \mathbb{R}^{n \times d}$ (a batch of n rows), hidden layer $Z_1 = XW_1 + b_1$, $H = \phi(Z_1)$, output logits $Z_2 = HW_2 + b_2$, softmax probabilities $P = \text{softmax}(Z_2)$, and mean cross-entropy loss over the batch

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \log P_{i, y_i}$$

where y_i is row i 's true class. **Backpropagation is nothing but the chain rule organized to reuse work**: compute $\partial \mathcal{L} / \partial Z_2$ once, then push it backward, layer by layer, caching nothing that isn't needed.

Step 1 — softmax + cross-entropy has a famously clean gradient. For one row with logits z and true class y : $\mathcal{L} = -z_y + \log \sum_k e^{z_k}$, so $\partial \mathcal{L} / \partial z_j = \text{softmax}(z)_j - [j = y]$ — in matrix form,

$$\frac{\partial \mathcal{L}}{\partial Z_2} = \frac{1}{n} (P - Y_{\text{onehot}})$$

"probabilities minus targets." This is the single most-asked derivation step in interviews; being able to produce it from $\log \sum \exp$ in two lines is the difference between memorized and understood. (The same form appears for sigmoid + binary cross-entropy: $\partial \mathcal{L} / \partial z = p - y$.)

Step 2 — through a linear layer. With $Z_2 = HW_2 + b_2$ and upstream gradient $G_2 = \partial \mathcal{L} / \partial Z_2$, the three consumers are

$$\frac{\partial \mathcal{L}}{\partial W_2} = H^T G_2, \quad \frac{\partial \mathcal{L}}{\partial b_2} = \sum_i (G_2)_i, \quad \frac{\partial \mathcal{L}}{\partial H} = G_2 W_2^T$$

The shapes dictate the formulas — $\partial\mathcal{L}/\partial W_2$ must match W_2 's shape, and $H^T G_2$ is the only way to combine the two available matrices into it. Shape-checking is a legitimate derivation tool; use it out loud in an interview.

Step 3 — through the activation. $H = \phi(Z_1)$ is elementwise, so the chain rule is an elementwise (Hadamard) product: $\partial\mathcal{L}/\partial Z_1 = (G_2 W_2^T) \odot \phi'(Z_1)$. For ReLU, ϕ' is the 0/1 mask of positive pre-activations; for tanh, $1 - \tanh^2(Z_1)$; for sigmoid, $\sigma(1 - \sigma)$. This single multiplication is where vanishing gradients enter: every layer multiplies the backward signal by ϕ' , and if $|\phi'| < 1$ chronically, depth compounds it geometrically.

Step 4 — repeat. $\partial\mathcal{L}/\partial W_1 = X^T(\partial\mathcal{L}/\partial Z_1)$, and so on down arbitrarily many layers: the backward pass is the same two moves — transpose-multiply through linear layers, mask-multiply through activations — applied in reverse order, reusing each layer's cached forward activations. Cost: one backward pass $\approx$ two forward passes; memory: all activations cached, which is why training memory, not compute, usually binds.

Trust nothing unverified. Listing 2 implements exactly this and then **gradient-checks** it: central finite differences $(\mathcal{L}(w+\epsilon) - \mathcal{L}(w-\epsilon))/2\epsilon$ against the analytic gradient give worst relative error 1.05×10^{-9} . Gradient checking is the professional habit for any hand-written backward pass (custom layers, custom losses); a relative error near 10^{-2} means a bug, near 10^{-7} or below means correct. The same listing then trains 2-64-64-3 on three-class spirals to 99.8% — the entire modern recipe (He init, ReLU, softmax + CE, full-batch gradient descent) in 60 lines of NumPy.

Activation functions

The activation decides what nonlinearity the network composes and, through its derivative, how gradients survive the backward pass. Listing 3 measures every claim below on 100,000 sampled pre-activations.

Sigmoid $\sigma(z) = 1/(1 + e^{-z})$ maps to $(0, 1)$ — the right *output* unit for binary probability, and mostly wrong everywhere else. Its derivative peaks at **0.25** (at $z = 0$) and dies in both tails ($\sigma'(3) \approx 0.045$), so every sigmoid hidden layer multiplies the backward signal by at most 0.25 — the arithmetic engine of vanishing gradients. It is also not zero-centered (outputs average 0.5), which biases downstream gradients toward same-sign updates and slows optimization. **Tanh** fixes the centering (outputs average ~ 0 , derivative peaks at 1.0) but keeps saturating tails — 13.4% of typical pre-activations already sit in its near-zero-gradient zone in Listing 3. Tanh was the pre-ReLU default and survives in gates (LSTMs, Chapter 15).

ReLU $\max(0, z)$ made depth practical: derivative exactly 1 on the positive side — no shrinkage, no saturation for active units — and it costs one comparison. Its two costs: outputs are not zero-centered, and the **dying ReLU** problem — a unit pushed to output 0 on *every* input has zero gradient and can never recover. Listing 3 stages the extreme case: one oversized bias update kills 100% of a layer's units permanently. **Leaky ReLU**

($0.01z$ for $z < 0$) keeps a trickle of gradient so no unit is unrecoverable — its measured dead-zone fraction is 0.0%. **GELU** $z\Phi(z)$ (used in BERT, GPT — Chapter 16's default) and **Swish/SiLU** $z\sigma(z)$ are smooth ReLU relatives: differentiable everywhere, slightly non-monotonic near zero (both dip below 0), empirically a consistent small win in transformers at negligible cost.

Selection rules an interviewer expects: hidden layers $\rightarrow$ ReLU by default, Leaky ReLU if you observe dead units, GELU in transformer-family models; output layer $\rightarrow$ sigmoid for binary probability, softmax for multiclass, identity for regression; never sigmoid/tanh in deep hidden stacks without a normalization scheme (Chapter 13's batch norm changed this calculus). And the "why nonlinear at all" answer — linear compositions collapse — should be reflexive.

Loss functions

The loss defines what the network optimizes; its *gradient* defines what the network actually feels. **MSE** $\frac{1}{n}\sum(\hat{y} - y)^2$ is the regression default (it is the negative log-likelihood under Gaussian noise — Chapter 5's story). Using it for classification through a sigmoid is the canonical mistake, and Listing 4(a) shows why: for a confidently wrong prediction ($p = 0.018$, $y = 1$), MSE's gradient through the sigmoid carries the factor $\sigma'(z) \approx 0.018$ and arrives at -0.035 , while **cross-entropy's** gradient is $p - y = -0.982$ — **27× larger exactly when the network most needs correcting**. Cross-entropy + sigmoid/softmax cancels the saturating derivative analytically; that cancellation is *the* reason for the pairing, and deriving it is a standard interview request. Cross-entropy is the MLE loss under a Bernoulli/categorical model (Chapter 1's MLE thread), and it is a proper scoring rule (Chapter 10).

Hinge loss $\max(0, 1 - m)$ on margins $m = y \cdot f(x)$ (labels ± 1) is the SVM loss (Chapter 6): zero once the margin exceeds 1 — confident-enough examples drop out of the gradient entirely — versus logistic loss which never quite reaches zero (Listing 4(c)). That zero zone yields sparse "support vector" behavior; logistic keeps pulling all points, hinge concentrates on the boundary. **Focal loss** $(1 - p_t)^\gamma \cdot \text{BCE}$ (Lin et al., RetinaNet) multiplies cross-entropy by a factor that vanishes for well-classified examples: Listing 4(b) shows an easy correct negative's loss crushed $0.051 \rightarrow 0.0001$ while a hard positive keeps most of its loss — the hard:easy ratio jumps from $23\times$ to $4,601\times$ at $\gamma = 2$. That is its purpose: extreme class imbalance (dense object detection with thousands of easy background windows per object), where plain BCE lets the easy majority drown the rare positives (Chapter 9's imbalance toolkit, continued).

Contrastive and triplet losses train *embeddings* rather than classifiers. Contrastive loss (on pairs) pulls same-class pairs together and pushes different-class pairs apart up to a margin; **triplet loss** $\max(0, \|a - p\|^2 - \|a - n\|^2 + \alpha)$ demands each anchor sit closer to a positive than to a negative by margin α . Listing 4(d) trains a linear embedding with triplet loss and watches geometry reorganize: the inter-centroid/intra-class ratio rises

from 0.98 (classes overlapping) to 3.52 (classes separated). This family powers face recognition (FaceNet), retrieval, and — as cousins of the InfoNCE loss — modern contrastive pretraining (SimCLR, CLIP; Chapters 17–18). The practical hinge: *mining* — random triplets quickly become easy and uninformative; semi-hard mining keeps the loss alive.

Weight initialization

Initialization looks like a detail and decides whether training starts at all. **Why zeros fail** is a symmetry argument, not a magnitude one: if all weights in a layer are equal, every hidden unit computes the same function, receives the same gradient, and takes the same update — they are *permanently identical*, so a 32-unit layer has the capacity of one unit. Listing 5(a) trains a zero-initialized network for 100 steps and counts **1 distinct column among 32** in W_1 . Random initialization exists to break this symmetry. (Biases can be zero; the weights' randomness already differentiates units.)

Magnitude then matters through signal propagation. For a linear-ish layer with fan-in m , the output variance of $\sum_j w_j x_j$ is $m\sigma_w^2 \text{Var}(x)$: too-small weights shrink the signal geometrically (Listing 5(b): $\mathcal{N}(0, 0.01)$ collapses activations to std 10^{-16} by layer 20), too-large weights rail tanh into saturation (std locks at 0.97 — saturated, zero gradient). **Xavier/Glorot initialization** $\sigma_w = \sqrt{1/m}$ (or $\sqrt{2/(m+n)}$ balancing forward and backward) sets $m\sigma_w^2 = 1$ to hold variance steady for symmetric activations like tanh — the probe shows healthy stds through all 20 layers. **He initialization** $\sigma_w = \sqrt{2/m}$ doubles the variance because ReLU zeroes half its inputs, halving variance per layer: Listing 5's punchline is Xavier-under-ReLU decaying to 10^{-3} by layer 20 while He holds std ≈ 1 . Rule: **Xavier for tanh/sigmoid, He for ReLU-family** — and remember that batch norm and residual connections (Chapter 13) were invented in part to make training robust to exactly these sensitivities.

Vanishing and exploding gradients

The backward pass multiplies the error signal by W_l^T and $\phi'(Z_l)$ at every layer — a product of many factors. If those factors are typically below 1 the gradient **vanishes** geometrically and early layers stop learning; above 1 it **explodes** and training diverges into NaNs. Listing 6 measures $\|\partial\mathcal{L}/\partial W_l\|$ across a 30-layer net: with sigmoid activations (max derivative 0.25) and Xavier init, the gradient decays from 10^{-1} at layer 30 to 10^{-20} at layer 1 — twenty orders of magnitude, layers 1–15 effectively frozen. ReLU with He init holds the ratio near 1 across all 30 layers. The same probe with He scaled $1.5\times$ hotter explodes to 10^4 – 10^5 — the knife-edge is real: propagation multipliers slightly below 1 vanish, slightly above explode, and depth is the exponent.

The remedy stack, in the order the field discovered it: **ReLU-family activations** (derivative 1 on the active side); **variance-preserving init** (Xavier/He, above);

normalization layers (batch/layer norm — recenter and rescale between layers; Chapter 13); **residual connections** ($h_{l+1} = h_l + f(h_l)$) gives gradients an identity highway — the enabler of 100+-layer nets; Chapters 13-14); and for recurrent nets, **gating** (LSTM/GRU; Chapter 15). Exploding gradients get the blunt instrument: **gradient clipping** — rescale the gradient when its global norm exceeds a cap, preserving direction. Listing 6's demo: norm 2979 $\rightarrow$ 5.0 with cosine similarity 1.0000 to the original direction. Clipping is standard in RNNs and transformer training; report the clip threshold with your hyperparameters, since it silently changes effective step sizes.

Code implementations

Listing 1 — The perceptron learns AND/OR, fails XOR; one hidden layer fixes it

```

"""Listing 1: the perceptron -- learns AND/OR, provably fails XOR; one hidden layer fixes it."""
import numpy as np

def train_perceptron(X, y, epochs=50, lr=0.1):
    w, b = np.zeros(X.shape[1]), 0.0
    for _ in range(epochs):
        for xi, yi in zip(X, y):
            pred = int(w @ xi + b > 0)
            w += lr * (yi - pred) * xi          # Rosenblatt update
            b += lr * (yi - pred)
    return w, b

X = np.array([[0,0],[0,1],[1,0],[1,1]], float)
for name, y in [("AND", np.array([0,0,0,1])),
                ("OR", np.array([0,1,1,1])),
                ("XOR", np.array([0,1,1,0]))]:
    w, b = train_perceptron(X, y)
    acc = ((X @ w + b > 0).astype(int) == y).mean()
    print(f"perceptron on {name}: accuracy {acc:.2f}    (w={np.round(w,2)}, b={b:+.2f})")

# 2-2-1 MLP with tanh hidden layer solves XOR
rng = np.random.default_rng(0)
W1, b1 = rng.normal(0, 1, (2, 2)), np.zeros(2)
W2, b2 = rng.normal(0, 1, (2, 1)), np.zeros(1)
yv = np.array([0,1,1,0], float).reshape(-1, 1)
for step in range(4000):
    H = np.tanh(X @ W1 + b1)                # hidden layer
    p = 1 / (1 + np.exp(-(H @ W2 + b2)))    # output sigmoid
    dz2 = (p - yv) / len(X)                 # dL/dz2 for BCE+sigmoid
    dW2, db2 = H.T @ dz2, dz2.sum(0)
    dz1 = (dz2 @ W2.T) * (1 - H**2)         # chain rule through tanh
    dW1, db1 = X.T @ dz1, dz1.sum(0)
    for P, G in [(W1,dW1), (b1,db1), (W2,dW2), (b2,db2)]: P -= 1.0 * G
    pred = (p > 0.5).astype(int).ravel()
    print(f"\nMLP (2-2-1, tanh) on XOR: accuracy {(pred == yv.ravel()).mean():.2f},"
          f" probs = {np.round(p.ravel(), 3)}")

```

Output:


```

perceptron on AND: accuracy 1.00    (w=[0.2 0.1], b=-0.20)
perceptron on OR:  accuracy 1.00    (w=[0.1 0.1], b=+0.00)
perceptron on XOR: accuracy 0.50    (w=[-0.1 0. ], b=+0.10)

MLP (2-2-1, tanh) on XOR: accuracy 1.00, probs = [0.001 0.999 0.999 0.   ]

```

The perceptron converges on the linearly separable AND/OR and never beats chance on XOR — no line separates its classes. Two tanh hidden units re-represent the inputs so the output unit's problem becomes linear: accuracy 1.00 with near-certain probabilities.

Listing 2 — A complete MLP with backprop, verified against numerical gradients

```

"""Listing 2: a complete MLP with backprop, verified against numerical gradients."""
import numpy as np
rng = np.random.default_rng(1)

def relu(z): return np.maximum(0, z)

def softmax(z):
    e = np.exp(z - z.max(axis=1, keepdims=True)) # subtract max: numerical stability
    return e / e.sum(axis=1, keepdims=True)

class MLP:
    def __init__(self, sizes):
        # e.g. [4, 8, 3]
        self.W = [rng.normal(0, np.sqrt(2/m), (m, n)) # He init
                   for m, n in zip(sizes[:-1], sizes[1:])]
        self.b = [np.zeros(n) for n in sizes[1:]]

    def forward(self, X):
        self.h = [X] # cache activations for backward
        for W, b in zip(self.W[:-1], self.b[:-1]):
            self.h.append(relu(self.h[-1] @ W + b))
        self.p = softmax(self.h[-1] @ self.W[-1] + self.b[-1])
        return self.p

    def loss(self, X, y):
        # mean cross-entropy
        p = self.forward(X)
        return -np.log(p[np.arange(len(y)), y] + 1e-12).mean()

    def backward(self, y):
        n = len(y)
        dz = self.p.copy(); dz[np.arange(n), y] -= 1; dz /= n # dL/dlogits = p -
onehot
        dW, db = [], []
        for l in range(len(self.W) - 1, -1, -1):
            dW.insert(0, self.h[l].T @ dz)
            db.insert(0, dz.sum(0))
            if l > 0:
                dz = (dz @ self.W[l].T) * (self.h[l] > 0) # through ReLU
        return dW, db

# gradient check: analytic vs central finite differences
X, y = rng.normal(size=(20, 4)), rng.integers(0, 3, 20)
net = MLP([4, 8, 3])
net.loss(X, y); dW, db = net.backward(y)
eps, worst = 1e-5, 0.0
for l in range(len(net.W)):
    for idx in [(0,0), (1,2), (net.W[l].shape[0]-1, net.W[l].shape[1]-1)]:
        keep = net.W[l][idx]
        net.W[l][idx] = keep + eps; lp = net.loss(X, y)
        net.W[l][idx] = keep - eps; lm = net.loss(X, y)

```

```

        net.W[l][idx] = keep
        num = (lp - lm) / (2 * eps)
        rel = abs(num - dW[l][idx]) / max(1e-12, abs(num) + abs(dW[l][idx]))
        worst = max(worst, rel)
    print(f"gradient check, worst relative error: {worst:.2e}    (< 1e-6 = correct
    backprop)")

# train on a real task: 3-class spirals
def spirals(n=900):
    k = n // 3; Xs, ys = [], []
    for c in range(3):
        t = np.linspace(0, 1, k); r = t * 2.5
        th = t * 4.5 + c * 2.09 + rng.normal(0, 0.12, k)
        Xs.append(np.c_[r*np.sin(th), r*np.cos(th)]); ys.append(np.full(k, c))
    return np.vstack(Xs), np.concatenate(ys)
Xs, ys = spirals()
net = MLP([2, 64, 64, 3])
for step in range(3001):
    loss = net.loss(Xs, ys); dW, db = net.backward(ys)
    for l in range(len(net.W)):
        net.W[l] -= 0.5 * dW[l]; net.b[l] -= 0.5 * db[l]
    if step % 1000 == 0:
        acc = (net.forward(Xs).argmax(1) == ys).mean()
        print(f"step {step:4d}: loss {loss:.4f}    train acc {acc:.3f}")

```

Output:

```

gradient check, worst relative error: 1.05e-09    (< 1e-6 = correct backprop)
step    0: loss 1.0474    train acc 0.420
step 1000: loss 0.0125    train acc 0.998
step 2000: loss 0.0091    train acc 0.998
step 3000: loss 0.0076    train acc 0.998

```

The backward pass is two moves repeated — transpose-multiply through linear layers, mask-multiply through ReLU — seeded by the softmax + CE gradient $P - Y$. Central-difference checking confirms it to 10^{-9} , and the same 60 lines train a 2-64-64-3 network to 99.8% on three-class spirals.

Listing 3 — Activation functions: gradients and dead zones measured

```

"""Listing 3: activation functions -- shapes, gradients, and dead/saturated zones
measured."""
import numpy as np
rng = np.random.default_rng(2)

acts = {
    "sigmoid":    (lambda z: 1/(1+np.exp(-z)),          lambda z: (s:=1/(1+np.exp(-
z)))*(1-s)),
    "tanh":       (np.tanh,                             lambda z: 1-np.tanh(z)**2),
    "ReLU":       (lambda z: np.maximum(0,z),           lambda z: (z>0).astype(float)),
    "LeakyReLU":  (lambda z: np.where(z>0,z,0.01*z),    lambda z: np.where(z>0,1,0.01)),
    "GELU":       (lambda z: z*0.5*(1+np.tanh(np.sqrt(2/np.pi)*(z+0.044715*z**3))),
    None),
    "Swish/SiLU": (lambda z: z/(1+np.exp(-z)),         None),
}
z = rng.normal(0, 2, 100_000) # typical pre-activation spread
print(f"{'activation':<12}{'max |grad|':>11}{'grad@z=3':>10}{'dead/saturated %':>18}
{'mean output':>13}")
for name, (f, g) in acts.items():
    if g is None: # numerical gradient
        eps = 1e-5; g = lambda z, f=f: (f(z+eps)-f(z-eps))/(2*eps)

```

```

    gr = g(z)
    frac = (np.abs(gr) < 0.01).mean() * 100          # near-zero gradient = no learning
    signal
    print(f"{name:<12}{np.abs(g(np.linspace(-6,6,1001))).max():>11.3f}"
          f"{float(np.mean(g(np.array([3.0])))):>10.4f}{frac:>17.1f}%
    {f(z).mean():>13.3f}")

# dying ReLU: one bad step can kill a unit permanently
W = rng.normal(0, 0.1, (100, 256)); X = rng.normal(size=(1000, 100))
b = np.zeros(256)
b -= 10.0                                           # simulate a huge gradient step on
biases
dead = (np.maximum(0, X @ W + b) == 0).all(axis=0).mean()
print(f"\nafter one oversized bias update: {dead*100:.0f}% of ReLU units output 0 on
ALL inputs")
print("gradient through a dead unit is 0 -> it can never recover (LeakyReLU keeps 0.01
alive)")

```

Output:

activation	max	grad	grad@z=3	dead/saturated %	mean output
sigmoid	0.250	0.0452		2.2%	0.500
tanh	1.000	0.0099		13.4%	-0.001
ReLU	1.000	1.0000		50.0%	0.795
LeakyReLU	1.000	1.0000		0.0%	0.787
GELU	1.129	1.0116		7.2%	0.711
Swish/SiLU	1.100	1.0881		1.5%	0.603

```

after one oversized bias update: 100% of ReLU units output 0 on ALL inputs
gradient through a dead unit is 0 -> it can never recover (LeakyReLU keeps 0.01 alive)

```

Sigmoid's gradient never exceeds 0.25 and is not zero-centered (mean output 0.5); tanh centers but saturates; ReLU passes gradient 1 when active but half of typical pre-activations sit in its zero zone, and the staged oversized update shows an entire layer dying irrecoverably — the case for Leaky ReLU's 0.01 lifeline.

Listing 4 — Loss functions: gradient behavior staged

```

"""Listing 4: loss functions -- MSE-vs-CE gradients, focal loss on imbalance, hinge,
triplet."""
import numpy as np
rng = np.random.default_rng(3)

# (a) why cross-entropy beats MSE for sigmoid outputs: gradient at a confident WRONG
answer
z = -4.0                                     # network says p=0.018, true label is 1
p = 1/(1+np.exp(-z))
print(f"confidently wrong (p={p:.3f}, y=1):")
print(f"  dMSE/dz = {2*(p-1)*p*(1-p):+.4f}  <- sigmoid derivative crushes the signal")
print(f"  dCE /dz = {p-1:+.4f}  <- full error, no saturation, 27x larger")

# (b) focal loss reweights easy examples away (class imbalance)
def bce(p, y):    return -(y*np.log(p) + (1-y)*np.log(1-p))
def focal(p, y, g=2.0):
    pt = p if y == 1 else 1-p
    return (1-pt)**g * bce(p, y)
p_easy, p_hard = 0.95, 0.30 # easy negative done right vs hard positive
print(f"\nbCE : easy-correct {bce(1-p_easy,0):.4f} hard-wrong {bce(p_hard,1):.4f}"
      f" ratio {bce(p_hard,1)/bce(1-p_easy,0):.0f}x")
print(f"focal: easy-correct {focal(1-p_easy,0):.4f} hard-wrong {focal(p_hard,1):.4f}"
      f" ratio {focal(p_hard,1)/focal(1-p_easy,0):.0f}x <- hard examples dominate")

```

```

# (c) hinge vs CE: hinge has a ZERO-loss zone -- confident-enough points stop mattering
margins = np.array([-1.0, 0.0, 0.5, 1.0, 2.0])
print(f"\nmargins      : {margins}")
print(f"hinge max(0,1-m): {np.maximum(0, 1-margins)}")
print(f"logistic log(1+e^-m): {np.round(np.log1p(np.exp(-margins)), 3)} <- never
exactly 0")

# (d) triplet loss learns an embedding where classes separate
X = rng.normal(size=(300, 10)); labels = rng.integers(0, 3, 300)
X += labels[:, None] * 0.8 # weak class signal
W = rng.normal(0, 0.3, (10, 2)) # linear embedding to 2-D
def dists(E):
    a = rng.integers(0, 300, 512)
    pos = np.array([rng.choice(np.where(labels == labels[i])[0]) for i in a])
    neg = np.array([rng.choice(np.where(labels != labels[i])[0]) for i in a])
    return a, pos, neg
for step in range(200):
    E = X @ W
    a, p_, n_ = dists(E)
    dp, dn = E[a]-E[p_], E[a]-E[n_]
    viol = (np.sum(dp**2,1) - np.sum(dn**2,1) + 1.0) > 0 # margin alpha=1
    grad = np.zeros_like(E)
    np.add.at(grad, a[viol], 2*(dp[viol] - dn[viol]))
    np.add.at(grad, p_[viol], -2*dp[viol])
    np.add.at(grad, n_[viol], 2*dn[viol])
    W -= 0.01 * (X.T @ grad) / viol.sum()
    if step in (0, 199):
        E = X @ W
        intra = np.mean([np.linalg.norm(E[labels==c] - E[labels==c].mean(0),
axis=1).mean() for c in range(3)])
        cents = np.array([E[labels==c].mean(0) for c in range(3)])
        inter = np.mean([np.linalg.norm(cents[i]-cents[j]) for i in range(3) for j in
range(i+1,3)])
        print(f"triplet step {step:3d}: intra-class spread {intra:.2f} vs inter-
centroid dist {inter:.2f}"
              f" (ratio {inter/intra:.2f})")

```

Output:

```

confidently wrong (p=0.018, y=1):
  dMSE/dz = -0.0347 <- sigmoid derivative crushes the signal
  dCE /dz = -0.9820 <- full error, no saturation, 27x larger

BCE : easy-correct 0.0513  hard-wrong 1.2040  ratio 23x
focal: easy-correct 0.0001  hard-wrong 0.5899  ratio 4601x <- hard examples dominate

margins      : [-1.  0.  0.5  1.  2. ]
hinge max(0,1-m): [2.  1.  0.5  0.  0. ]
logistic log(1+e^-m): [1.313 0.693 0.474 0.313 0.127] <- never exactly 0
triplet step   0: intra-class spread 1.01 vs inter-centroid dist 1.00 (ratio 0.98)
triplet step 199: intra-class spread 0.44 vs inter-centroid dist 1.55 (ratio 3.52)

```

Four lessons in one run: cross-entropy's gradient is 27× MSE's exactly where correction matters most; focal loss turns a 23× hard:easy loss ratio into 4,601×; hinge's zero zone retires confident examples while logistic never does; and triplet loss reorganizes an embedding from overlapping (ratio 0.98) to separated (3.52).

Listing 5 — Initialization: symmetry and signal propagation

```

"""Listing 5: weight initialization -- zeros never break symmetry; Xavier/He keep
signals alive."""
import numpy as np
rng = np.random.default_rng(4)

# (a) zero init: every hidden unit stays IDENTICAL forever
X = rng.normal(size=(64, 10)); y = rng.integers(0, 2, 64).astype(float).reshape(-1,1)
W1, b1 = np.zeros((10, 32)), np.zeros(32)
W2, b2 = np.zeros((32, 1)), np.zeros(1)
for _ in range(100):
    H = np.tanh(X @ W1 + b1)
    p = 1/(1+np.exp(-(H @ W2 + b2)))
    dz2 = (p - y)/len(X); dW2 = H.T @ dz2
    dz1 = dz2 @ W2.T * (1 - H**2); dW1 = X.T @ dz1
    W1 -= 0.5*dW1; W2 -= 0.5*dW2; b1 -= 0.5*dz1.sum(0); b2 -= 0.5*dz2.sum(0)
print(f"zero init after 100 steps: distinct columns in W1 = "
      f"{len(np.unique(np.round(W1, 10), axis=1)[0])} of 32 <- all units learned the
SAME thing")

# (b) activation statistics through a 20-layer net under three inits
def probe(scale_fn, act, name, gain_desc):
    h = rng.normal(size=(512, 256))
    stds = []
    for l in range(20):
        fan_in = h.shape[1]
        W = rng.normal(0, scale_fn(fan_in), (fan_in, 256))
        h = act(h @ W)
        stds.append(h.std())
    print(f"{name:<28} layer1 std {stds[0]:.3f}   layer10 {stds[9]:.3e}   layer20
{stds[19]:.3e}   {gain_desc}")

print("\nsignal propagation, tanh network:")
probe(lambda f: 0.01, np.tanh, " N(0, 0.01) too small", "-> signal dies")
probe(lambda f: 1.0, np.tanh, " N(0, 1) too big", "-> saturation
(std ~= 1 tanh rails)")
probe(lambda f: np.sqrt(1/f), np.tanh, " Xavier 1/sqrt(fan_in)", "-> stable")
print("signal propagation, ReLU network:")
probe(lambda f: np.sqrt(1/f), lambda z: np.maximum(0,z), " Xavier under ReLU", "->
halves each layer")
probe(lambda f: np.sqrt(2/f), lambda z: np.maximum(0,z), " He sqrt(2/fan_in)", "->
stable")

```

Output:

```

zero init after 100 steps: distinct columns in W1 = 1 of 32 <- all units learned the
SAME thing

signal propagation, tanh network:
  N(0, 0.01) too small      layer1 std 0.156   layer10 1.039e-08   layer20 1.099e-16
-> signal dies
  N(0, 1) too big          layer1 std 0.975   layer10 9.741e-01   layer20 9.741e-01
-> saturation (std ~= 1 tanh rails)
  Xavier 1/sqrt(fan_in)     layer1 std 0.627   layer10 2.226e-01   layer20 1.608e-01
-> stable
signal propagation, ReLU network:
  Xavier under ReLU         layer1 std 0.586   layer10 3.176e-02   layer20 1.260e-03
-> halves each layer
  He sqrt(2/fan_in)         layer1 std 0.819   layer10 8.610e-01   layer20 1.110e+00
-> stable

```

Zeros never break symmetry — 32 units collapse to 1 distinct weight column after 100 steps of training. The propagation probes show each regime: too-small init starves the signal by layer 10, too-large rails tanh, Xavier holds tanh steady, and ReLU needs He's factor of 2 because it discards half the variance per layer.

Listing 6 — Vanishing and exploding gradients, measured; clipping

```

"""Listing 6: vanishing and exploding gradients measured layer-by-layer on the backward pass."""
import numpy as np
rng = np.random.default_rng(5)

def backward_norms(depth, width, act, dact, w_std):
    """Forward through `depth` layers, then backprop a unit error; return grad norm per layer."""
    Ws, hs, zs = [], [rng.normal(size=(128, width))], []
    for _ in range(depth):
        W = rng.normal(0, w_std(width), (width, width)); Ws.append(W)
        z = hs[-1] @ W; zs.append(z); hs.append(act(z))
    dz = np.ones((128, width)) / (128*width) # unit upstream error
    norms = []
    for l in range(depth-1, -1, -1):
        dz = dz * dact(zs[l]) # through activation
        norms.append(np.linalg.norm(hs[l].T @ dz)) # ||dL/dW_l||
        dz = dz @ Ws[l].T
    return norms[::-1]

sig = lambda z: 1/(1+np.exp(-z)); dsig = lambda z: sig(z)*(1-sig(z))
relu = lambda z: np.maximum(0,z); drelu = lambda z: (z>0).astype(float)

for name, act, dact, std in [
    ("sigmoid, Xavier init", sig, dsig, lambda w: np.sqrt(1/w)),
    ("ReLU, He init", relu, drelu, lambda w: np.sqrt(2/w)),
    ("ReLU, He x 1.5 (too hot)", relu, drelu, lambda w: 1.5*np.sqrt(2/w)),
]:
    ns = backward_norms(30, 128, act, dact, std)
    print(f"{name} ||grad W|| layer30 {ns[29]:.2e} layer15 {ns[14]:.2e} layer1 {ns[0]:.2e}"
          f" ratio L1/L30 {ns[0]/ns[29]:.1e}")

# gradient clipping: cap the global norm, keep the direction
g = rng.normal(0, 30, 10_000) # an exploded gradient vector
gn = np.linalg.norm(g); cap = 5.0
g_clip = g * min(1.0, cap/gn)
print(f"\nglobal-norm clipping: ||g|| {gn:.1f} -> {np.linalg.norm(g_clip):.1f}"
      f", direction preserved (cos = {g @ g_clip/(gn*np.linalg.norm(g_clip)):.4f})")

```

Output:

```

sigmoid, Xavier init      ||grad W|| layer30 1.16e-01 layer15 5.31e-11 layer1
1.10e-20 ratio L1/L30 9.5e-20
ReLU, He init             ||grad W|| layer30 1.23e+00 layer15 2.30e+00 layer1
4.72e-01 ratio L1/L30 3.8e-01
ReLU, He x 1.5 (too hot)  ||grad W|| layer30 7.67e+04 layer15 1.34e+05 layer1
2.70e+04 ratio L1/L30 3.5e-01

global-norm clipping: ||g|| 2979.5 -> 5.0, direction preserved (cos = 1.0000)

```

Thirty sigmoid layers attenuate the gradient twenty orders of magnitude — layers 1–15 are frozen. ReLU + He holds the ratio near 1; scale He by 1.5 and gradients sit at 10^5 . Clipping caps the global norm at 5.0 while keeping the exact direction.

Listing 7 — Universal approximation in practice

```

"""Listing 7: universal approximation in practice -- one hidden layer fits an arbitrary function."""
import numpy as np
rng = np.random.default_rng(6)

f_true = lambda x: np.sin(3*x) + 0.5*np.sign(x) + 0.3*x**2      # wiggly +
discontinuous
X = np.linspace(-2, 2, 400).reshape(-1, 1)
y = f_true(X.ravel()).reshape(-1, 1)

def fit_1hidden(width, steps=6000, lr=0.02):
    W1 = rng.normal(0, 3, (1, width)); b1 = rng.uniform(-2, 2, width) * -W1.ravel()
    W2 = rng.normal(0, 0.1, (width, 1)); b2 = np.zeros(1)
    for _ in range(steps):
        H = np.tanh(X @ W1 + b1)
        p = H @ W2 + b2
        d = 2*(p - y)/len(X)
        dW2, db2 = H.T @ d, d.sum(0)
        dh = d @ W2.T * (1 - H**2)
        dW1, db1 = X.T @ dh, dh.sum(0)
        W1 -= lr*dW1; b1 -= lr*db1; W2 -= lr*dW2; b2 -= lr*db2
    H = np.tanh(X @ W1 + b1)
    return H @ W2 + b2

print("one hidden layer, tanh -- MSE vs width:")
for w in [2, 8, 32, 128]:
    p = fit_1hidden(w)
    print(f"width {w:4d}: MSE {np.mean((p - y)**2):.4f}")
print("\nUAT: error -> 0 as width grows; the theorem promises existence,"

"\nnot that SGD finds it, and says nothing about sample efficiency or depth being
cheaper.")

```

Output:

```

one hidden layer, tanh -- MSE vs width:
width    2: MSE 0.1210
width    8: MSE 0.0219
width   32: MSE 0.0071
width  128: MSE 0.0065

UAT: error -> 0 as width grows; the theorem promises existence,
not that SGD finds it, and says nothing about sample efficiency or depth being cheaper.

```

One tanh hidden layer drives approximation error down monotonically with width on a wiggly, discontinuous target — the theorem lives. The plateau from 32 to 128 is the practical footnote: capacity stops being the bottleneck and optimization/data take over.

Listing 8 — End-to-end: the scratch MLP on real digits

```

"""Listing 8: end-to-end -- the Listing 2 MLP on real data (digits), with a train/test split."""

```



```

import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
rng = np.random.default_rng(7)

digits = load_digits()
X = digits.data / 16.0 # scale pixels to [0,1]
Xtr, Xte, ytr, yte = train_test_split(X, digits.target, test_size=0.25,
                                     random_state=0, stratify=digits.target)

def relu(z): return np.maximum(0, z)
def softmax(z):
    e = np.exp(z - z.max(axis=1, keepdims=True)); return e / e.sum(axis=1,
keepdims=True)

sizes = [64, 128, 64, 10]
W = [rng.normal(0, np.sqrt(2/m), (m, n)) for m, n in zip(sizes[:-1], sizes[1:])]
b = [np.zeros(n) for n in sizes[1:]]

def forward(Xb):
    hs = [Xb]
    for Wl, bl in zip(W[:-1], b[:-1]): hs.append(relu(hs[-1] @ Wl + bl))
    return hs, softmax(hs[-1] @ W[-1] + b[-1])

BATCH, EPOCHS, LR0 = 64, 30, 0.1
for ep in range(EPOCHS):
    LR = LR0 * 0.93**ep # simple exponential decay (Ch. 13
                        # does better)
    order = rng.permutation(len(Xtr))
    for s in range(0, len(Xtr), BATCH): # mini-batch SGD
        idx = order[s:s+BATCH]
        hs, p = forward(Xtr[idx])
        dz = p.copy(); dz[np.arange(len(idx)), ytr[idx]] -= 1; dz /= len(idx)
        for l in range(len(W)-1, -1, -1):
            dWl, dbl = hs[l].T @ dz, dz.sum(0)
            if l > 0: dz = (dz @ W[l].T) * (hs[l] > 0)
            W[l] -= LR*dWl; b[l] -= LR*dbl
    if ep % 10 == 9 or ep == 0:
        _, ptr = forward(Xtr); _, pte = forward(Xte)
        loss = -np.log(ptr[np.arange(len(ytr)), ytr] + 1e-12).mean()
        print(f"epoch {ep+1:2d}: train loss {loss:.4f} "
              f"train acc {(ptr.argmax(1)==ytr).mean():.4f} test acc "
              f"{(pte.argmax(1)==yte).mean():.4f}")

```

Output:

```

epoch 1: train loss 1.6664 train acc 0.3898 test acc 0.4022
epoch 10: train loss 0.2015 train acc 0.9525 test acc 0.9511
epoch 20: train loss 0.1626 train acc 0.9592 test acc 0.9533
epoch 30: train loss 0.1171 train acc 0.9740 test acc 0.9600

```

The chapter's machinery assembled — He init, ReLU, softmax + cross-entropy, mini-batch SGD with a simple decay schedule — reaches 96.0% test accuracy on the digits dataset in NumPy alone. Every deep learning framework automates exactly this; none of it changes.

Pitfalls, comparisons and practical tips

Activation selection:

Activation	Range	Max gradient	Zero-centered	Main risk	Use
Sigmoid	$(0,1)$	0.25	No	Vanishing gradients	Binary output layer only
Tanh	$(-1,1)$	1.0	Yes	Saturating tails	Gates (LSTM/GRU); small nets
ReLU	$[0,\infty)$	1.0	No	Dying units	Hidden-layer default
Leaky ReLU	$(-\infty,\infty)$	1.0	$\sim$ No	—	When dead units observed
GELU / Swish	$(-0.17,\infty)$ -ish	~ 1.1	$\sim$ No	Slightly costlier	Transformers

Loss selection: regression $\rightarrow$ MSE (Gaussian noise) or MAE/Huber (outliers, Chapter 10); binary classification $\rightarrow$ BCE with sigmoid; multiclass $\rightarrow$ CE with softmax; extreme imbalance $\rightarrow$ focal or class weights; margin/sparse-support behavior $\rightarrow$ hinge; metric learning/embeddings $\rightarrow$ contrastive, triplet (with hard mining), InfoNCE.

Initialization: He for ReLU family, Xavier for tanh/sigmoid, never all-zeros for weights (biases fine). If forward activations' std collapses or saturates across layers, fix init before touching anything else.

The recurring pitfalls:

- **Forgetting why nonlinearity exists.** Any stack of purely linear layers is one linear layer. If asked what a 100-layer linear network can represent: exactly a single matrix multiply.
- **MSE on classification through sigmoid.** Gradient carries $\sigma'(z)$ and dies precisely on confident mistakes (Listing 4's $27\times$ gap). Pair sigmoid/softmax with cross-entropy; the saturating term cancels.
- **Unverified hand-written backprop.** Always gradient-check custom layers against central differences (Listing 2's 10^{-9}); relative error 10^{-2} = bug.
- **Reading the UAT as a guarantee.** Existence $\neq$ findability $\neq$ generalization. It licenses none of "one wide layer is all you need."
- **Ignoring dead units.** Monitor the fraction of always-zero ReLU outputs; tens of percent dead = lower the learning rate or switch to Leaky/GELU.
- **Symmetric init "just this once."** Constant-value init of any kind (not just zeros) collapses a layer's units to clones.
- **Diagnosing vanishing/exploding blind.** Log per-layer gradient norms (Listing 6 is 15 lines); the signature — early layers frozen, or norms in the thousands — tells you which remedy applies (init/activation/normalization vs clipping).
- **Numerically naive softmax/log.** Subtract the max before exponentiating; add an epsilon inside logs. NaNs at high learning rates usually trace here or to explosion.

- **Forgetting the cache.** Backprop needs forward activations; training memory scales with $\text{depth} \times \text{batch}$, which is why gradient checkpointing exists (Chapter 29).

Interview questions and answers

Q1. Why can't a single perceptron solve XOR, and what exactly fixes it?

A perceptron is a thresholded linear function, so its decision boundary is a hyperplane; XOR's positive points (0,1),(1,0) and negative points (0,0),(1,1) are not separable by any line. A hidden layer with a nonlinearity fixes it by re-representing the input: two hidden units compute two half-plane indicators whose combination is linearly separable for the output unit. Listing 1: perceptron stuck at 0.50, 2-2-1 tanh MLP at 1.00. The general statement: hidden layers learn a representation in which the problem becomes linear.

Q2. State the universal approximation theorem precisely, and give three things it does NOT say.

A feedforward network with one hidden layer, enough units, and any non-polynomial activation can approximate any continuous function on a compact set to arbitrary accuracy. It does not say: (1) gradient descent will find those weights — it's an existence result; (2) how many units are needed — width can grow exponentially with input dimension; (3) anything about generalizing from finite data. Nor does it make depth pointless: some functions need exponentially fewer parameters when expressed deep.

Q3. Derive the gradient of softmax + cross-entropy with respect to the logits.

For logits z and true class y , $L = -\log \text{softmax}(z)_y = -z_y + \log \sum_k e^{z_k}$. Then $\partial L / \partial z_j = -[j=y] + e^{z_j} / \sum_k e^{z_k} = p_j - [j=y]$: probabilities minus one-hot target. Batch matrix form: $(P - Y)/n$. The identical shape appears for sigmoid+BCE: $\partial L / \partial z = p - y$. This cancellation of the softmax/sigmoid derivative is why the pairing exists.

Q4. Walk through backprop for a two-layer MLP: what are the gradients for W_2 , b_2 , W_1 ?

Forward: $Z_1 = XW_1 + b_1$, $H = \phi(Z_1)$, $Z_2 = HW_2 + b_2$, $P = \text{softmax}(Z_2)$, $L = \text{mean CE}$. Backward: $G_2 = \partial L / \partial Z_2 = (P - Y)/n$; then $\partial L / \partial W_2 = H^T G_2$, $\partial L / \partial b_2 = \text{column-sum of } G_2$, $\partial L / \partial H = G_2 \cdot W_2^T$; through the activation $\partial L / \partial Z_1 = (G_2 \cdot W_2^T) \odot \phi'(Z_1)$; finally $\partial L / \partial W_1 = X^T (\partial L / \partial Z_1)$. Two moves repeat for any depth: transpose-multiply through linear layers, elementwise-multiply through activations, reusing cached forward activations.

Q5. How do you verify a hand-written backward pass, and what error magnitude indicates a bug?

Gradient checking: compare the analytic gradient with central finite differences $(L(w+\epsilon) - L(w-\epsilon))/2\epsilon$ at sampled parameters ($\epsilon \approx 1e-5$), using relative error $|a-n|/(|a|+|n|)$. Below $\sim 1e-6$ correct; around $1e-4$ suspicious (or a non-differentiable kink); $1e-2$ or worse a bug. Listing 2 lands at $1e-9$. Use double precision and disable stochastic layers (dropout) while checking.

Q6. Why must activation functions be nonlinear?

Composition of linear maps is linear: $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$. A 100-layer purely linear network has exactly the expressive power of one linear layer — it can only represent linear functions. The nonlinearity between layers is what lets depth compose increasingly complex functions.

Q7. Compare sigmoid, tanh, and ReLU as hidden activations.

Sigmoid: range (0,1), max gradient 0.25, not zero-centered — both saturation and centering problems; obsolete for hidden layers. Tanh: zero-centered, max gradient 1, but tails still saturate — better, still vanishing-prone deep. ReLU: gradient exactly 1 when active, no saturation on the positive side, trivially cheap — the modern default; costs are non-centered outputs and dying units. Numbers from Listing 3: grad at $z=3$ is 0.045 (sigmoid), 0.0099 (tanh), 1.0 (ReLU).

Q8. What is the dying ReLU problem and how do you detect and fix it?

A unit whose pre-activation is negative for every input outputs 0 always and receives zero gradient — permanently dead. Cause: a large gradient step (often high LR) pushing bias/weights far negative; Listing 3 kills 100% of a layer with one oversized bias update. Detect: track the fraction of units with all-zero activations over a validation batch. Fix: lower the learning rate, Leaky ReLU/GELU (nonzero gradient everywhere), or He init to start in a healthy regime.

Q9. Why do GELU/Swish sometimes beat ReLU, and where are they standard?

Both are smooth (differentiable everywhere — no kink at 0), keep a small gradient for negative inputs (no hard death), and are slightly non-monotonic near zero, which appears to help optimization in very deep attention stacks. GELU $z\Phi(z)$ is standard in BERT/GPT-family transformers; Swish $z\sigma(z)$ was found by architecture search and is common in EfficientNet-family CNNs. Gains are consistent but small; cost is negligible on accelerators.

Q10. Why is cross-entropy preferred over MSE for classification? Show the gradient argument.

With sigmoid output $p = \sigma(z)$: MSE gradient $dL/dz = 2(p-y)\cdot\sigma'(z)$, and $\sigma'(z) \rightarrow 0$ exactly when the network is confident — including confidently wrong. CE gradient $dL/dz = p - y$: the sigmoid derivative cancels, leaving the raw error. Listing 4: at $p=0.018$ with $y=1$, MSE's gradient is -0.035 , CE's -0.982 — $27\times$ larger where correction is most needed. MSE for classification also makes the loss non-convex in z and corresponds to the wrong likelihood (Gaussian instead of Bernoulli).

Q11. Explain focal loss: formula, mechanism, and when to use it.

$FL = (1-p_t)^\gamma \cdot CE$, where p_t is the predicted probability of the true class and $\gamma \approx 2$. The modulating factor $(1-p_t)^\gamma \approx 0$ for well-classified examples, so the loss (and gradient) budget concentrates on hard cases. Listing 4: easy-correct example's loss falls $0.051 \rightarrow 0.0001$ while a hard-wrong keeps $\sim$ half, moving the hard:easy ratio from $23\times$ to $4,601\times$. Use under extreme class imbalance with abundant easy negatives — canonical case dense object detection (RetinaNet); an alternative to resampling/class weights (Chapter 9).

Q12. Hinge vs logistic loss — what behavioral difference does hinge's zero region create?

Hinge $\max(0, 1 - m)$ is exactly zero once margin $m \geq 1$: those examples leave the gradient, so the solution depends only on boundary points — support-vector sparsity. Logistic $\log(1 + e^{-m})$ is positive for all finite margins, so every example keeps pulling (Listing 4: at $m=2$, hinge 0.0 vs logistic 0.127). Consequences: hinge gives sparse, margin-focused solutions but no probabilities; logistic gives calibrated-ish probabilities but no sparsity.

Q13. Define triplet loss, explain the margin, and why mining matters.

$L = \max(0, \|f(a) - f(p)\|^2 - \|f(a) - f(n)\|^2 + \alpha)$: anchor a must be closer to positive p than negative n by at least margin α , else gradient pulls a, p together and pushes a, n apart. The margin prevents the trivial collapse "make everything equidistant." Mining: random triplets are quickly satisfied (zero loss, zero learning); semi-hard mining selects negatives inside the margin so the loss stays informative. Listing 4 raises inter/intra separation $0.98 \rightarrow 3.52$. Applications: FaceNet, retrieval, and the conceptual ancestor of contrastive pretraining losses.

Q14. Why exactly does initializing all weights to zero fail? Is it a magnitude problem?

No — symmetry. If every weight in a layer is identical, all units compute identical outputs, receive identical gradients, and update identically forever: an n -unit layer has the capacity of 1 unit. Listing 5 trains 100 steps from zeros and finds 1 distinct weight column of 32. Any constant init fails the same way. Random init breaks the symmetry; zero *biases* are fine because random weights already differentiate units.

Q15. Derive the Xavier initialization rule and say what He changes and why.

For $z = \sum_j w_j x_j$ with m independent inputs, $\text{Var}(z) = m \cdot \sigma_w^2 \cdot \text{Var}(x)$. Keeping $\text{Var}(z) = \text{Var}(x)$ across layers requires $m \cdot \sigma_w^2 = 1$, i.e. $\sigma_w = \sqrt{1/m}$ (Glorot's version balances backward too: $2/(m+n)$). ReLU zeroes half its inputs, cutting the variance roughly in half per layer, so He compensates with $\sigma_w = \sqrt{2/m}$. Listing 5: Xavier under ReLU decays activations to $1e-3$ std by layer 20; He holds ~ 1 . Rule: Xavier for tanh/sigmoid, He for ReLU family.

Q16. What causes vanishing gradients, mechanically?

Backprop multiplies the error signal by $\phi'(Z_l)$ and W_l^T at each layer — a product of dozens of factors. If typical factor magnitude < 1 (sigmoid's $\phi' \leq 0.25$ is the classic culprit; ill-scaled weights too), the product shrinks geometrically with depth: Listing 6 measures a 30-layer sigmoid net attenuating the gradient by $1e-20$, freezing layers 1–15. Symptoms: early-layer weights barely change, loss plateaus high, deeper stacking makes results worse.

Q17. List the remedies for vanishing gradients in the order you'd apply them.

(1) ReLU-family activation — derivative 1 on the active path. (2) Variance-preserving init — He/Xavier matched to activation. (3) Normalization layers — batch/layer norm recenters and rescales between layers (Chapter 13). (4) Residual connections — $h + f(h)$ provides an identity gradient path, enabling 100+ layers (Chapter 13). (5) For recurrence, gated cells (LSTM/GRU, Chapter 15). Each attacks the multiplicative decay differently: bigger factors, well-scaled factors, re-normalized signals, or an additive bypass.

Q18. How does gradient clipping work, and why clip the norm rather than each element?

Global-norm clipping: if $\|g\| > c$, replace g with $g \cdot c / \|g\|$ — magnitude capped, direction exactly preserved (Listing 6: $2979 \rightarrow 5.0$, cosine 1.0000). Elementwise clipping changes the direction (large coordinates saturate, small ones don't), effectively optimizing a different objective. Norm clipping is the standard guard for RNN and transformer training against rare exploding batches; the threshold is a real hyperparameter — too low silently shrinks all steps.

Q19. Why is the backward pass roughly the cost of two forward passes, and what dominates training memory?

Each layer's backward needs two matmuls (gradient w.r.t. weights: $h^T \cdot g$; gradient w.r.t. input: $g \cdot W^T$) versus one in forward — hence $\sim 2\times$. Memory is dominated not by weights but by cached forward activations, which scale with batch $\times$ width $\times$ depth, needed for those backward matmuls. That is why large-model training is memory-bound and why gradient checkpointing trades recompute for memory (Chapter 29).

Q20. What's the difference between a loss surface being non-convex and being untrainable? Why does SGD work anyway?

Neural losses are non-convex with exponentially many critical points, so no global-optimum guarantee exists — yet in practice most local minima of over-parameterized nets have similar, good loss, and the real obstacles are saddle points and plateaus, which SGD's noise helps escape. Width helps: more parameters create connected low-loss regions. The honest statement: we lack a complete theory; empirically, over-parameterization + SGD + good init/normalization reliably finds low-loss solutions that also generalize (regularization discussion, Chapter 13).

Q21. Your deep network's training loss is stuck near its initial value. Give a debugging sequence.

(1) Overfit a tiny subset (10 examples) — if it can't, the pipeline/gradients are broken, not the capacity. (2) Check per-layer gradient norms — all ~ 0 early layers means vanishing (activation/init); NaN/huge means explosion (LR, clipping). (3) Verify loss at init $\approx \log(k)$ for k classes — much higher indicates bad init or missing softmax-stability. (4) Check data: shuffled labels, unscaled inputs. (5) Gradient-check any custom layer. (6) Sweep the LR by $10\times$ both ways. Ordered, falsifiable steps are what the interviewer wants.

Q22. Softmax overflows for large logits. How is it computed stably, and why is the answer unchanged?

Subtract the row max before exponentiating: $\text{softmax}(z) = \text{softmax}(z - \max z)$, because $e^{\{z_j - c\}} / \sum e^{\{z_k - c\}} = e^{\{z_j\}} / \sum e^{\{z_k\}}$ for any constant c . With $c = \max z$ the largest exponent is $e^0 = 1$ — no overflow; pair with log-sum-exp for the loss and an epsilon inside logs. Listing 2's implementation does exactly this.

Q23. When would you still choose tanh over ReLU today?

Where bounded, zero-centered outputs are structurally required: gates and cell states in LSTMs/GRUs (a gate must live in $(0,1)$ or $(-1,1)$), attention-free recurrent dynamics where unbounded activations destabilize the recurrence, small shallow networks where vanishing isn't binding, and outputs that must lie in $(-1,1)$ (e.g. normalized control signals, DCGAN generator output). For deep feedforward hidden stacks, ReLU-family wins.

Q24. Explain forward-pass shapes: batch X is $n \times d$, layer widths $d \rightarrow h \rightarrow k$. Give every matrix shape through backprop.

Forward: $W1$ $d \times h$, $b1$ h ; $Z1 = XW1 + b1$ is $n \times h$; H $n \times h$; $W2$ $h \times k$; $Z2 = HW2 + b2$ $n \times k$; P $n \times k$. Backward: $G2 = (P - Y)/n$ is $n \times k$; $dW2 = H^T G2$ $h \times k$ (matches $W2$); $db2$ k ; $dH = G2 W2^T$ $n \times h$; $dZ1 = dH \odot \phi'(Z1)$ $n \times h$; $dW1 = X^T dZ1$ $d \times h$; $db1$ h . Shape agreement uniquely determines where transposes go — a reliable way to reconstruct backprop under pressure.

Q25. What does "pre-activation" vs "activation" mean, and which does each formula need?

Pre-activation $Z = XW + b$ (before the nonlinearity); activation $H = \phi(Z)$ (after). Backprop through the nonlinearity needs $\phi'(Z)$ evaluated at the *pre-activation* (for ReLU the mask $Z > 0$, equivalently $H > 0$); the weight gradient needs the *activation* of the previous layer ($H^T \cdot G$). Mixing them up is the most common scratch-implementation bug — caching both per layer, as Listing 2 does via h , avoids it.

Q26. MSE corresponds to Gaussian noise. What noise model does MAE correspond to, and cross-entropy?

Maximum likelihood correspondences: MSE $\leftrightarrow$ Gaussian noise (minimizer: conditional mean); MAE $\leftrightarrow$ Laplace noise (minimizer: conditional median — hence outlier robustness); binary cross-entropy $\leftrightarrow$ Bernoulli likelihood; categorical CE $\leftrightarrow$ multinomial. Choosing a loss = choosing a noise/label model, which connects Chapter 1's MLE, Chapter 5's GLMs, and Chapter 10's proper scoring rules into one story — a synthesis interviewers reward.

Q27. Why do residual connections help gradients, in one equation?

With $h_{l+1} = h_l + f(h_l)$, the local Jacobian is $I + \partial f / \partial h_l$, so the backward signal is $g \cdot (I + \partial f / \partial h)^T = g + g \cdot (\partial f / \partial h)^T$ — an identity term flows regardless of what f does. Gradients reach early layers additively rather than through a pure product of layer Jacobians, breaking the geometric decay of Listing 6. This single additive path is what made 100+-layer networks trainable (ResNet; Chapter 14).

Q28. You must implement and train an MLP with no framework. Enumerate the pieces and the order you'd build/verify them.

(1) Stable softmax + CE loss with known init value $\log(k)$. (2) Forward pass with cached activations. (3) Backward pass: $(P-Y)/n$ seed, transpose-multiplies, activation masks. (4) Gradient check to $<1e-6$ before any training. (5) He init (given ReLU), zero biases. (6) Mini-batch SGD loop with shuffling, LR chosen by short sweep, simple decay. (7) Sanity ladder: overfit 10 samples → train/val curves → test once. Listings 2 and 8 are this recipe; on digits it reaches 96.0% test — and every framework merely automates these exact steps.